

Extending QuAK with Nested Quantitative Automata

Thomas A. Henzinger¹, Nicolas Mazzocchi², N. Ege Sarac³, and Harun Yilmaz⁴

¹ Institute of Science and Technology Austria (ISTA), Austria
tah@ista.ac.at

² Slovak University of Technology in Bratislava, Slovak Republic
nicolas.mazzocchi@stuba.sk

³ CISA Helmholtz Center for Information Security
ege.sarac@cisa.de

⁴ Sabanci University
harun.yilmaz@sabanciuniv.edu

Abstract. Quantitative automata (QAs) extend finite-state ω -automata with weighted transitions to specify quantitative system properties. However, their finite weight sets rule out properties like average response time, where response times can be arbitrarily large. Nested quantitative automata (NQAs) overcome this limitation: a parent automaton spawns child automata to compute unbounded values over finite subwords and aggregates them into a final result. Despite this expressiveness, NQAs have lacked practical tool support to date.

We address this gap by extending the Quantitative Automata Kit (QuAK), a software tool for QA analysis, to support NQAs. Our core contribution is implementing a suite of flattening procedures that reduce NQAs to QAs, leveraging QuAK’s existing decision procedures. The tool handles all decidable combinations of parent aggregators (including limits and averages) and child functions (extrema and monotonic or bounded summations). Experiments on response-time and resource-consumption benchmarks demonstrate QuAK’s effectiveness.

1 Introduction

Formal verification has traditionally focused on boolean properties: a system either satisfies a specification or it does not. Yet many critical requirements are inherently quantitative; we may require that a server’s average workload stays above 20%, or that a process’s peak memory usage stays under 1GB. Quantitative automata (QAs) [4] address this need by assigning rational weights to transitions in finite-state ω -automata and aggregating them via value functions such as Sup , LimSup , or LimSupAvg . The Quantitative Automata Kit (QuAK) [2,3] provides the first comprehensive tool support for analyzing such automata, implementing algorithms for emptiness, inclusion, and safety-liveness analyses.

However, QAs cannot express a fundamental class of quantitative properties. Consider *average response time*: a system receives requests and issues grants,

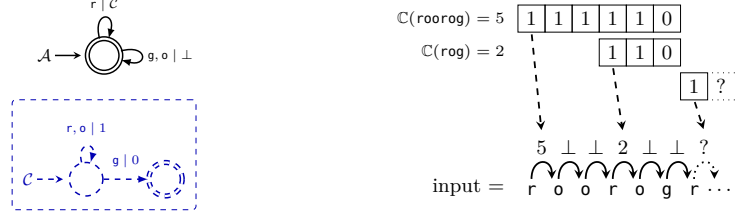


Fig. 1: A $(\text{LimSupAvg}, \text{Sum}^+)$ -automaton $\mathbb{A} = (\mathcal{A}, \mathcal{C})$ for average response time. *Left:* Parent $\mathcal{A}$ spawns child $\mathcal{C}$ on each request. Child $\mathcal{C}$ accumulates weight 1 per step until a grant, then terminates. *Right:* On the shown prefix, the spawned instances of $\mathcal{C}$ return 5 and 2; the third instance is still active. Silent transitions (marked $\perp$) do not contribute to the parent’s value, so the running average after the event g is $\frac{7}{2}$.

and we wish to measure the long-run average delay between each request and the following grant. Since a QA has a finite set of transition weights, every run produces values within a bounded range while response times can be arbitrarily large. Therefore, no QA can express such unbounded per-event measurements [7].

Nested quantitative automata (NQAs) [7] solve this problem. A *parent* automaton reads an infinite word and spawns *child* automata on designated transitions; each child runs over a finite subword and returns a value upon termination. The parent aggregates these values using its value function. Figure 1 shows a $(\text{LimSupAvg}, \text{Sum}^+)$ -automaton $\mathbb{A} = (\mathcal{A}, \mathcal{C})$ that computes average response time. The parent $\mathcal{A}$ spawns the child $\mathcal{C}$ on every request; other letters do not contribute a value. The child $\mathcal{C}$ accumulates weight 1 on each step until the first grant, then terminates with the accumulated sum. On the input prefix $\text{roorogr} \dots$, the first child returns 5 (waiting five steps for its grant) and the second returns 2, while the third is still running. The parent’s LimSupAvg aggregates these returned values into their long-run average. Because children can run arbitrarily long, their return values are unbounded, enabling measurement of quantities that non-nested automata cannot express.

Despite solid theoretical foundations and positive decidability results [7,5,6], there has been no practical implementation of NQA algorithms. We bridge this gap by extending QuAK to support NQAs. In particular:

1. We extend QuAK with a modular architecture for NQA analysis built around *flattening*: translating an NQA problem instance into a QA one whose answer coincides with the original query.
2. We implement flattening procedures for all emptiness and universality problems known to be decidable for combinations of parent aggregators $\{\text{Sup}, \text{Inf}, \text{LimSup}, \text{LimInf}, \text{LimSupAvg}, \text{LimInfAvg}\}$ and child aggregators $\{\text{Min}, \text{Max}, \text{Sum}^B, \text{Sum}^+, \text{Sum}^-\}$.
3. We evaluate QuAK on two benchmark families (response time and resource consumption), showing its scalability across a range of NQA types and sizes.

Related work. NQAs were introduced in [7], which established their expressiveness beyond QAs and characterized much of the decidability landscape.

The bounded-width NQAs [5] restrict the number of concurrently active children, which yields better complexity in several cases. Quantitative monitor automata [6] provide a counter-based formulation expressively equivalent to bounded-width NQAs. QuAK [2,3] is the first tool to automate the analysis of QAs; we build directly on its codebase. Weighted automata tools such as Vaucanson [16], Vcsn [8], and Awali [15] primarily target finite words with semiring-style semantics. Signal temporal logic tools (e.g., Breach [9], S-TaLiRo [1], and RTAMT [17]), timed-automata and hybrid-systems tools (e.g., UPPAAL [14] and HyTech [12]), and probabilistic model checkers (e.g., PRISM [13] and Storm [11]) address orthogonal quantitative verification problems.

2 Theoretical Background

Quantitative Automata and Value Functions A *quantitative automaton* (QA) over (finite or infinite) words is a tuple $\mathcal{A} = (\Sigma, Q, q_0, \delta, F, C)$ consisting of a finite alphabet Σ , a finite state set Q , an initial state q_0 , a transition relation $\delta \subseteq Q \times \Sigma \times Q$, an accepting-state set $F \subseteq Q$, and a weight function $C : \delta \rightarrow \mathbb{Q}$.

A *run* of $\mathcal{A}$ on a (finite or infinite) word $w = w_1 w_2 \dots$ is a sequence of states $\pi = q_0 q_1 \dots$ with $(q_{i-1}, w_i, q_i) \in \delta$ for all $i \geq 1$. Each run induces a weight sequence $C(\pi) = C(q_0, w_1, q_1), C(q_1, w_2, q_2), \dots$. A *value function* aggregates the weight sequence of a run into a single value. For infinite sequences, we consider:

$$\begin{aligned} \text{Inf}(x) &= \inf_{i \geq 1} x_i & \text{Sup}(x) &= \sup_{i \geq 1} x_i \\ \text{LimInf}(x) &= \liminf_{i \rightarrow \infty} x_i & \text{LimSup}(x) &= \limsup_{i \rightarrow \infty} x_i \\ \text{LimInfAvg}(x) &= \liminf_{k \rightarrow \infty} \frac{1}{k} \sum_{i=1}^k x_i & \text{LimSupAvg}(x) &= \limsup_{k \rightarrow \infty} \frac{1}{k} \sum_{i=1}^k x_i \end{aligned}$$

For finite sequences $x = x_1 \dots x_n$, we consider $\text{Min}(x) = \min_i x_i$, $\text{Max}(x) = \max_i x_i$, $\text{Sum}^+(x) = \sum_{i=1}^n |x_i|$, $\text{Sum}^-(x) = -\sum_{i=1}^n |x_i|$, and $\text{Sum}^B(x)$, which equals $\sum_{i=1}^n x_i$ if all partial sums remain in $[-B, B]$ for a fixed bound B , and the first bound crossed ($-B$ or B) otherwise. For a QA $\mathcal{A}$ on infinite (resp. finite) words with value function f , a run π is *accepting* if it visits F infinitely often (resp. if it ends at a state in F), and the *value* of $\mathcal{A}$ on a word w is the supremum of $f(C(\pi))$ over all accepting runs π on w . An automaton is *deterministic* if each state has at most one outgoing transition per letter.

Nested Quantitative Automata A *nested quantitative automaton* (NQA) [7] $\mathbb{A} = (\mathcal{A}, \mathcal{B}_1, \dots, \mathcal{B}_k)$ consists of a *parent automaton* $\mathcal{A} = (\Sigma, Q, q_0, \delta, F, L)$, where $(\Sigma, Q, q_0, \delta, F)$ is an automaton over infinite words and $L : \delta \rightarrow \{0, 1, \dots, k\}$ is a labeling function, and k *child automata* $\mathcal{B}_1, \dots, \mathcal{B}_k$, each a QA over finite words. A parent transition t with $L(t) = j > 0$ *invokes* (or *spawns*) child $\mathcal{B}_j$; a transition with $L(t) = 0$ is *silent* and invokes no child. An NQA is an (f, g) -*automaton* if the parent uses value function f and every child uses value function g .

A *run* of $\mathbb{A}$ on $w \in \Sigma^\omega$ is a tuple $(\Pi, \pi_1, \pi_2, \dots)$ where $\Pi = \Pi[0]\Pi[1]\dots$ is a run of the parent on $w = w_1 w_2 \dots$, and for each $i \geq 1$: if the i -th parent transition has label $j > 0$, then π_i is a run of $\mathcal{B}_j$ on some finite subword $w_i \dots w_{i'}$; otherwise π_i is undefined. When π_i is defined, the child invoked at position i

	$f \in \{\text{Inf}, \text{Sup}, \text{LimInf}, \text{LimSup}\}$		$f \in \{\text{LimInfAvg}, \text{LimSupAvg}\}$	
	Emptiness	Universality	Emptiness	Universality
$g \in \{\text{Min}, \text{Max}, \text{Sum}^B\}$	PSPACE	EXPSpace	PSPACE	Undecidable
$g \in \{\text{Sum}^+, \text{Sum}^-\}$	PSPACE	EXPSpace	EXPSp./Open [†]	Undecidable

Table 1: Complexity of decision problems for NQAs with parent aggregator f and child aggregator g [7]. [†](LimInfAvg, Sum⁺) is open, the rest is in EXPSpace.

terminates at position i' , returning the value $g(C(\pi_i))$. The run $(\Pi, \pi_1, \pi_2, \dots)$ is *accepting* if: (i) the parent run Π visits F infinitely often, (ii) every invoked child run terminates in an accepting state, and (iii) infinitely many parent transitions are non-silent. The *value* of an accepting run is f applied to the sequence of returned child values, omitting silent transitions. The value of A on w is the supremum over all accepting runs on w . An NQA is *deterministic* if the parent and all children are deterministic, and accepting states in each child have no outgoing transitions. Figure 1 shows a deterministic (LimSupAvg, Sum⁺)-automaton computing average response time.

Decision Problems Given a QA or NQA and a threshold $\lambda \in \mathbb{Q}$, the *emptiness* problem asks whether there exists $w \in \Sigma^\omega$ with value at least λ , and the *universality* whether every $w \in \Sigma^\omega$ has value at least λ . Table 1 summarizes the complexity landscape from [7]. QuAK supports all decidable combinations.

Algorithmic Overview The decision procedures reduce NQA problems to QA problems via *flattening*: constructing a QA that simulates the NQA by tracking active child states. We give the key ideas below; see [7] for the details.

(1) *Regular children.* When children use value functions $g \in \{\text{Min}, \text{Max}, \text{Sum}^B\}$, the set of possible return values is finite, so for each value v , the words of value v form a regular language. This enables a reduction from (f, g) -automata to *equivalent* f -automata with silent transitions [7, Lem. 4.10]: for each child invocation guess each child’s return value, verify the guess via a DFA recognizing the corresponding language, and emit the guessed value as the transition weight. The construction incurs exponential blowup in the total size of these verifying DFAs.

(2) *Threshold-based bounding.* For NQAs with $f \in \{\text{Sup}, \text{LimSup}, \text{Inf}, \text{LimInf}\}$ and $g \in \{\text{Sum}^+, \text{Sum}^-\}$, child return values exceeding a given threshold λ can be truncated without affecting the emptiness or universality decision [7, Thm. 4.18], reducing to the Sum^B case and enabling the regular children construction.

(3) *Extremal parents with monotonic children.* For emptiness checking of NQAs with $f \in \{\text{Sup}, \text{LimSup}, \text{Inf}, \text{LimInf}\}$ and $g \in \{\text{Min}, \text{Max}, \text{Sum}^+, \text{Sum}^-\}$, we provide specialized guess-and-verify procedures that exploit two properties to achieve smaller state spaces than the general reduction. First, the child functions are *monotonic*: for Min and Max, we track the closest value to a guessed return value seen so far; for Sum⁺ and Sum[−], the guessed value acts as a budget to produce or consume. This enables pruning configurations where children can no longer achieve their guessed return values. Second, the parent functions are *extremal*: they select a single weight rather than computing an aggregate like an average. The parent objective determines how many guessed values to track. For

Inf and LimInf, the flattened automaton pairs each child state with a guessed return value, tracking all active children; states with dominated guesses are pruned. For Sup and LimSup, exceeding the threshold requires witnessing only one child return value, so we track a guessed value for one distinguished child at a time while other children are tracked only for termination. This yields substantially smaller state spaces for Sup and LimSup.

(4) *Limit-average with unbounded children.* Recall that the universality problem for limit-average QAs is undecidable, and this extends to NQAs with any child aggregator; we therefore focus on emptiness. For NQAs with $f \in \{\text{LimInfAvg}, \text{LimSupAvg}\}$ and $g = \text{Sum}^-$, the construction proceeds in three phases [7, Thm. 4.20]. First, the automaton is determinized via alphabet extension, encoding each combination of nondeterministic choices into a new input letter. Second, child steps are synchronized so that all active children advance simultaneously. Third, a powerset-based flattening tracks bounded-multiplicity configurations. For emptiness of $(\text{LimSupAvg}, \text{Sum}^+)$ -automata, we first check whether child return values can grow unboundedly; if so, emptiness holds trivially, otherwise the problem reduces to the Sum^B case [7, Lem. 5.10]. The case of $(\text{LimInfAvg}, \text{Sum}^+)$ -automata remains open; unlike LimSupAvg, the unboundedness check does not apply since unbounded child returns do not resolve emptiness for infimum-based objectives.

(5) *Silent-weight elimination.* The procedures above produce QAs with silent transitions, arising from parent $\perp$ -labels that do not invoke children. These must be eliminated to obtain standard QAs. For extremal objectives, silent transitions are replaced by neutral weights (e.g., $-\infty$ for Sup and LimSup, $+\infty$ for Inf and LimInf) that do not affect the computed value [7, Lem. 4.6]. For limit-average objectives we can eliminate silent weights via the path-compression construction of [7, Lem. 4.7], which replaces each maximal silent segment by a single shortcut transition, possibly increasing the number of transitions.

(6) *Antichain-based universality.* We decide universality for NQAs by an on-the-fly search for a counterexample word, pruning the exploration using an antichain as in FORKLIFT [10]. For the quantitative setting, contexts are enriched with value summaries: in addition the reachable state set, we store for each state the current maximal weight bound, and extend subsumption relation to compare both reachability and these bounds [2]. To incorporate Büchi acceptance in this version of QuAK, each context keeps two reachability summaries: all reachable pairs, and those reachable via a path that already visited an accepting state. We only evaluate candidate cycles against the weight threshold once acceptance progress has been witnessed. See Appendix A for details.

3 QuAK’s Architecture and Implementation

Overview QuAK [2,3] is a C++ library and command-line tool for analyzing QAs. We extend QuAK to support NQAs. The key addition is a suite of *flattening* procedures that compile an NQA into a QA with silent weights. This approach supports decidable emptiness and universality checks of all combinations of parent objectives in $\{\text{Sup}, \text{Inf}, \text{LimSup}, \text{LimInf}, \text{LimSupAvg}, \text{LimInfAvg}\}$

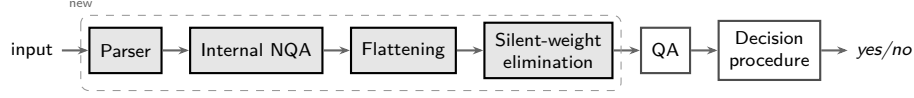


Fig. 2: The extended architecture of QuAK. Shaded components are new in this version. A nested quantitative automaton (NQA) is flattened into a standard quantitative automaton (QA) and fed to existing decision procedures.

with child objectives in $\{\text{Min}, \text{Max}, \text{Sum}^B, \text{Sum}^+, \text{Sum}^-\}$. Figure 2 depicts the pipeline; the rest of this section describes each phase and its implementation.

Input Format and Internal Representation QuAK reads automata from plain text files as transition lists $a : v, q \rightarrow p$ (letter a , weight v , from state q to p), with the initial state being the source of the first transition. We extend this format with an optional `final` directive for Büchi acceptance; otherwise, all states are accepting. Internally, a QA is stored as an **Automaton** object that does not fix a value function, allowing the same structure to be analyzed under different objectives. Automaton states are integer-indexed, and transitions are stored in adjacency-list form with both outgoing and incoming edges. While constructing an **Automaton** object, QuAK prunes unreachable states and caches its SCCs [2]. QuAK requires automata to be *complete*: every state must have at least one outgoing transition per letter. Determinism is not required, but QuAK detects it and dispatches to specialized routines when available.

Input NQAs consist of one `@PARENT` block and one `@CHILD  $i$` block per child. Children use the syntax above, while parent transitions use $a : i, q \rightarrow p$, where $i > 0$ invokes child i and $i = 0$ denotes a silent transition. The parent uses Büchi acceptance, and children use finite-word acceptance. Internally, the **NestedAutomaton** and **ChildAutomaton** classes extend QuAK’s base **Automaton** class. A **NestedAutomaton** object has a parent automaton of type **Automaton** and a vector of pointers to **ChildAutomaton** objects.

Flattening Nested Quantitative Automata The flattening module implements the constructions; see Section 2 for the algorithmic details.

(1) *Regular children.* For NQAs with $g \in \{\text{Min}, \text{Max}, \text{Sum}^B\}$, we implement a general flattening procedure. For each child invocation, we guess the return value and verify the guess by tracking the child’s states in parallel; verified guesses emit the guessed value as a transition weight. Rather than explicitly constructing verifier DFAs, we track each guess on-the-fly: an active guess records a child index, the guessed value, and a set of child states (the nondeterministic frontier). Child states that cannot reach a final state with the guessed value are pruned using a reachability table precomputed via reverse BFS. The set of possible return values per child is refined by analyzing the parent–child product restricted to parent states from which some accepting state is reachable. QuAK uses this procedure for universality across all parent aggregators f , and for emptiness when $f \in \{\text{LimInfAvg}, \text{LimSupAvg}\}$ or $g = \text{Sum}^B$; specialized procedures below handle the remaining emptiness cases.

(2) *Threshold-based bounding.* For universality of NQAs with $f \in \{\text{Sup}, \text{LimSup}, \text{Inf}, \text{LimInf}\}$ and $g \in \{\text{Sum}^+, \text{Sum}^-\}$, we truncate child values at the

threshold and apply the Sum^B flattening procedure above. Although this also applies to emptiness, we use the specialized procedures below.

(3) *Extremal parents with monotonic children.* For emptiness of NQAs with $f \in \{\text{Sup}, \text{LimSup}, \text{Inf}, \text{LimInf}\}$ and $g \in \{\text{Min}, \text{Max}, \text{Sum}^+, \text{Sum}^-\}$, we implement specialized flattening procedures. For $g \in \{\text{Min}, \text{Max}\}$, each flattened state stores the parent state, a Büchi-tracking bit, and four bitvectors indexed by child state (one for each combination of guessed outcome and current threshold status). For $g \in \{\text{Sum}^+, \text{Sum}^-\}$, each child-state entry instead stores a budget in $[0, |\lambda|]$, where λ is the emptiness threshold. Exploration proceeds via an explicit work-list; discovered states are deduplicated via a hash-table lookup before being enqueued. Since each child state may have multiple outgoing edges, we enumerate all combinations via backtracking: for each child edge we update the destination array, recurse to the next child, then restore the entry before trying the next edge. For $f \in \{\text{Sup}, \text{LimSup}\}$, we track one nondeterministically selected distinguished token with its full budget while summarizing all other active tokens with presence bitvectors alone; this suffices because the supremum requires only a single witness. The output automaton uses binary weights; spawning a child emits 1 if the guess is that the child will cross the threshold, and 0 otherwise.

(4) *Limit-average with unbounded children.* For emptiness of NQAs with $f = \text{LimSupAvg}$ and $g = \text{Sum}^+$, the unboundedness check is done as a $(\text{LimSup}, \text{Sum}^+)$ -emptiness check with a threshold λ capturing the largest value a single child can accumulate without forcing a repetition of some global configuration with a strictly larger accumulated sum. The bounded case then invokes the regular flattening above using Sum^B with $B = \lambda$. For $f \in \{\text{LimInfAvg}, \text{LimSupAvg}\}$ and $g = \text{Sum}^-$, we implement the three-step flattening. The multiset component of each powerset state is encoded as a sparse sorted vector of $(\text{state-id}, \text{count})$ pairs, containing only the states with nonzero multiplicity. State deduplication uses a hash table keyed by $(\text{parent-state}, \text{sparse-vector})$ pairs. Exploration proceeds by BFS with on-the-fly state generation. The theoretical configuration c bound is computed upfront, and instances exceeding the multiplicity cap are pruned.

(5) *Silent-weight elimination.* Before passing the flattened automaton to QuAK's existing decision procedures, we eliminate transitions with silent weights. For prefix-independent objectives $(\text{LimSup}, \text{LimInf}, \text{LimSupAvg}, \text{LimInfAvg})$, we can reduce the potential overhead by only considering those within accepting SCCs. This optimization is enabled by default, but can be deactivated.

(6) *Antichains, interfacing, and optimizations.* The flattening functions are composed internally to implement emptiness and universality checks, but are also exposed in the public API for use in custom analysis pipelines. We also extended two of QuAK's algorithms for QAs to improve its performance and interface with NQAs. For limit-average emptiness, rather than computing maximum mean cycles over the entire automaton, we decompose by SCC, restrict to accepting SCCs. For FORKLIFT inclusion [10], we handle Büchi acceptance by augmenting the context to track two relations (all reachable pairs, and pairs reachable through an accepting state) and requiring cycles to visit an accepting state before checking weight thresholds.

Availability and Usage QuAK is open-source under the MIT license and available online.¹ The tool has no external dependencies other than a C++17 compiler. Instructions and examples are provided in the repository’s README.

4 Experimental Evaluation

We evaluate QuAK’s new capabilities along two benchmarks: (i) *response-time* properties and (ii) *resource-consumption* constraints.

Setup. All experiments were run on Ubuntu 24.04.3 with an Intel Ultra 7 Processor 255U and 32 GB RAM. The tool was compiled with GCC using C++17 and optimization level `-O3`. Each instance was limited to 300 seconds and 30 GB of memory. Reported runtimes are wall-clock seconds averaged over three runs, excluding parsing time; each measurement was preceded by a warm-up run.

4.1 Response Time

We study a parametric family of NQAs $\mathbb{A}_{n,k}$ for bounded-response properties in request-grant protocols. The parameter n bounds the number of simultaneously pending requests, and k bounds the allowable response time. The alphabet comprises a request r , a grant g , and a neutral action o . For $n \geq 1$ and $k \geq n$, the parent automaton has $\frac{n(2k-n+1)}{2} + 2$ states and uses a single two-state child automaton. A child instance is spawned on each request, so at most n instances are concurrently active; if this bound is exceeded or some request goes ungranted for more than k steps, the parent moves to a rejecting sink. Each child accumulates the number of steps between its spawning request and the matching grant, and the parent value function determines how these per-request response times are combined: **Sup** yields the worst-case response time, while **LimSupAvg** yields the long-run average. Figure 3 in Appendix B shows a representative instance $\mathbb{A}_{2,2}$.

Unlike the unbounded response-time automaton in Figure 1, the bounds enforced by $\mathbb{A}_{n,k}$ yield a finite-state benchmark suitable for systematic experiments. This family also captures a standard verification scenario: given a finite-state model of a server, quantify the worst-case or average response time over all permitted behaviors. We evaluate QuAK on this benchmark with emptiness of $(\text{Sup}, \text{Sum}^+)$ -automata, emptiness of $(\text{LimSupAvg}, \text{Sum}^+)$ -automata, and universality of $(\text{Sup}, \text{Sum}^B)$ -automata. The results are reported in Table 2.

Emptiness of $(\text{Sup}, \text{Sum}^+)$ -automata. Table 2a shows that emptiness checking scales well on this family: the dependence on k is close to quadratic, while the dependence on n is comparatively mild. This behavior is consistent with the specialized procedure for extremal parents with monotonic children (Section 2) and with the structure of $\mathbb{A}_{n,k}$. All children are copies of the same two-state deterministic automaton, so before any grant occurs they differ only in accumulated budget, not control state. The non-distinguished part of the construction therefore tracks essentially one child configuration regardless of n ; the dependence on n arises from the parent’s enumeration of queue states.

¹ We plan to submit it for artifact evaluation.

(a) $(\text{Sup}, \text{Sum}^+)$ emptiness									(b) $(\text{LimSupAvg}, \text{Sum}^+)$ emptiness										(c) $(\text{Sup}, \text{Sum}^B)$ universality								
$n \backslash k$	4	8	16	32	64	128	256	512	$n \backslash k$	3	4	5	6	7	8	9	10		$n \backslash k$	2	3	4	5	6	7	8	9
4	.00	.00	.01	.01	.06	.32	1.5	6.3	3	.00	.00	.00	.01	.02	.05	.13	.31		2	.00	.00	.01	.04	.14	.42	1.2	3.6
8	-	.00	.01	.03	.16	.76	3.3	14	4	-	.00	.00	.01	.07	.28	1.1	3.2		3	-	.00	.02	.08	.40	2.3	12	44
16	-	-	.01	.07	.39	1.7	7.3	32	5	-	-	.00	.03	.21	1.3	5.5	20		4	-	-	.02	.12	1.1	12	159	t
32	-	-	-	.13	.85	3.8	17	72	6	-	-	-	.03	.29	2.8	18	93		5	-	-	-	.14	2.0	51	t	t
64	-	-	-	-	1.3	7.8	35	m	7	-	-	-	-	.33	4.3	39	284		6	-	-	-	-	2.4	136	t	m
128	-	-	-	-	-	12	71	m	8	-	-	-	-	-	4.7	58	m		7	-	-	-	-	-	177	t	m
256	-	-	-	-	-	-	m	m	9	-	-	-	-	-	-	63	m		8	-	-	-	-	-	-	t	m
512	-	-	-	-	-	-	-	m	10	-	-	-	-	-	-	-	m		9	-	-	-	-	-	-	-	m

Table 2: Response time benchmarks: runtime in seconds, m = out of memory (30 GB), t = out of time (300 s).

Emptiness of $(\text{LimSupAvg}, \text{Sum}^+)$ -automata. Table 2b shows a different pattern: runtime grows moderately with k but rapidly with n , as expected for this type of automata [7,5]. The construction here has two sources of state-space growth. First, the regular-children procedure guesses up to k candidate return values and maintains $\Theta(k)$ verifying-DFA states for each, yielding $\Theta(k^2)$ states from this component alone. Second, concurrency multiplies this cost since handling n simultaneously active instances requires tracking sets of active guesses. Since concurrency is capped at n , only combinations of at most n guesses are explored, but increasing n remains the dominant cost driver; it determines the exponent, whereas k affects only the base.

Universality of $(\text{Sup}, \text{Sum}^B)$ -automata. Table 2c shows that universality follows a similar pattern: runtime grows rapidly in both parameters, with n dominant. This is expected, since both experiments operate on the same flattened automaton; however, universality is consistently more expensive because it must handle the flattened automaton’s nondeterminism, requiring a PSPACE procedure rather than the PTIME emptiness check.

4.2 Resource Consumption

We study another parametric family of NQAs $\mathbb{B}_{n,k}$ that monitors resource consumption in a system with dynamically started and terminated processes. The parameter n bounds the number of concurrently running processes, and k bounds the number of distinct resources. The alphabet comprises $n(k+2)$ actions: for each process $i \in \{1, \dots, n\}$, a start action s_i , a termination action t_i , and k resource-access actions $a_{i,1}, \dots, a_{i,k}$. For $n \geq 1$ and $k \geq 1$, the parent automaton has a single control state and uses n child templates $B_1, \dots, B_n$, each with $2^k + 3$ states. When process i starts, the parent spawns an instance of child B_i , which remains active until the matching termination t_i . While active, the instance tracks which resources process i accesses; upon termination, it returns the number of distinct resources used, computed via the **Max** value function. The child rejects the words that violate the protocol, such as starting an already-running process. The parent aggregates per-process values with either **Sup**, yielding the maximal consumption across executions, or **LimSupAvg**, yielding the long-run average. Figure 4 in Appendix B shows a representative instance $\mathbb{B}_{1,2}$.

This family provides a controlled benchmark with two orthogonal scaling axes: increasing k multiplies the distinct resource-usage patterns a process can

(a) (Sup,Max) emptiness							(b) (LimSupAvg,Max) emptiness					
$n \backslash k$	1	2	3	4	5	6	$n \backslash k$	1	2	3	4	
1	.00	.00	.00	.00	.00	.00	1	.00	.00	.00	.00	
2	.00	.00	.01	.06	.37	1.9	2	.00	.03	1.1	62	
3	.01	.10	1.1	11	103	m	3	.12	48	t	t	
4	.12	2.4	43	m	m	m	4	20	t	t	m	
5	1.3	38	m	m	m	m						
6	9.9	m	m	m	m	m						

Table 3: Resource consumption benchmarks: runtime in seconds, m = out of memory (30 GB), t = out of time (300 s).

exhibit, while increasing n multiplies the concurrent overlap between processes. Although $\mathbb{B}_{n,k}$ is deterministic as a nested automaton, any equivalent non-nested automaton must encode resource subsets explicitly and hence grows exponentially in the size of $\mathbb{B}_{n,k}$ [7]. We evaluate QuAK on emptiness of (Sup, Max)- and (LimSupAvg, Max)-automata; results appear in Table 3.

Emptiness of (Sup, Max)-automata. Table 3a shows rapid blowup in both parameters. Each child encodes which subset of resources a process has accessed, giving $\Theta(2^k)$ control states per process. Since at most one instance of each process can be active at a time, the joint configuration space has $2^{\Theta(nk)}$ elements. Although the specialized procedure tracks a guessed value for only one distinguished child, it must still carry the full set of active configurations in the background to ensure every spawned instance eventually terminates. In contrast, the response-time children yield only $O(k)$ budget values during flattening, giving $2^{O(n \log k)}$ joint configurations, which explains the gentler scaling in Table 2a.

Emptiness of (LimSupAvg, Max)-automata. As Table 3b shows, the limit-average case is substantially harder than the supremum even at small sizes. The specialized procedure for Sup tracks a guessed value for only one distinguished child at a time, but the regular-children procedure for LimSupAvg must track guesses for all active children simultaneously. This combines the $2^{\Theta(nk)}$ configuration space with an additional factor for per-child guesses, pushing the construction beyond resource budgets at relatively small parameter values.

5 Conclusion

We presented the first practical tool for analyzing nested quantitative automata (NQAs), extending QuAK with flattening procedures that reduce nested emptiness and universality to their non-nested counterparts. This enables automated verification of quantitative properties, such as average response time, that standard quantitative automata cannot express. Two directions merit further investigation. On the practical side, symbolic state representations could mitigate the exponential blowup in configuration space of the flattened automata. On the theoretical side, identifying tractable fragments that preserve expressiveness for common specification patterns would broaden the tool’s applicability.

References

1. Annpureddy, Y., Liu, C., Fainekos, G., Sankaranarayanan, S.: S-taliro: A tool for temporal logic falsification for hybrid systems. In: Abdulla, P.A., Leino, K.R.M. (eds.) *Tools and Algorithms for the Construction and Analysis of Systems - 17th International Conference, TACAS 2011, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2011, Saarbrücken, Germany, March 26-April 3, 2011. Proceedings. Lecture Notes in Computer Science*, vol. 6605, pp. 254–257. Springer (2011). https://doi.org/10.1007/978-3-642-19835-9_21
2. Chalupa, M., Henzinger, T.A., Mazzocchi, N., Saraç, N.E.: Quak: Quantitative automata kit. In: Margaria, T., Steffen, B. (eds.) *Leveraging Applications of Formal Methods, Verification and Validation. Software Engineering Methodologies - 12th International Symposium, ISoLA 2024, Crete, Greece, October 27-31, 2024, Proceedings, Part IV. Lecture Notes in Computer Science*, vol. 15222, pp. 3–20. Springer (2024). https://doi.org/10.1007/978-3-031-75387-9_1, https://doi.org/10.1007/978-3-031-75387-9_1
3. Chalupa, M., Henzinger, T.A., Mazzocchi, N., Saraç, N.E.: Automating the analysis of quantitative automata with quak. In: Gurfinkel, A., Heule, M. (eds.) *Tools and Algorithms for the Construction and Analysis of Systems - 31st International Conference, TACAS 2025, Held as Part of the International Joint Conferences on Theory and Practice of Software, ETAPS 2025, Hamilton, ON, Canada, May 3-8, 2025, Proceedings, Part I. Lecture Notes in Computer Science*, vol. 15696, pp. 303–312. Springer (2025). https://doi.org/10.1007/978-3-031-90643-5_16, https://doi.org/10.1007/978-3-031-90643-5_16
4. Chatterjee, K., Doyen, L., Henzinger, T.A.: Quantitative languages. *ACM Trans. Comput. Log.* **11**(4), 23:1–23:38 (2010). <https://doi.org/10.1145/1805950.1805953>
5. Chatterjee, K., Henzinger, T.A., Otop, J.: Nested weighted limit-average automata of bounded width. In: Faliszewski, P., Muscholl, A., Niedermeier, R. (eds.) *41st International Symposium on Mathematical Foundations of Computer Science, MFCS 2016, Kraków, Poland, August 22-26, 2016. LIPIcs*, vol. 58, pp. 24:1–24:14. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2016). <https://doi.org/10.4230/LIPIcs.MFCS.2016.24>, <https://doi.org/10.4230/LIPIcs.MFCS.2016.24>
6. Chatterjee, K., Henzinger, T.A., Otop, J.: Quantitative monitor automata. In: Rival, X. (ed.) *Static Analysis - 23rd International Symposium, SAS 2016, Edinburgh, UK, September 8-10, 2016, Proceedings. Lecture Notes in Computer Science*, vol. 9837, pp. 23–38. Springer (2016). https://doi.org/10.1007/978-3-662-53413-7_2, https://doi.org/10.1007/978-3-662-53413-7_2
7. Chatterjee, K., Henzinger, T.A., Otop, J.: Nested weighted automata. *ACM Trans. Comput. Log.* **18**(4), 31:1–31:44 (2017). <https://doi.org/10.1145/3152769>
8. Demaille, A., Duret-Lutz, A., Lombardy, S., Sakarovitch, J.: Implementation concepts in vacanson 2. In: Konstantinidis, S. (ed.) *Implementation and Application of Automata - 18th International Conference, CIAA 2013, Halifax, NS, Canada, July 16-19, 2013. Proceedings. Lecture Notes in Computer Science*, vol. 7982, pp. 122–133. Springer (2013). https://doi.org/10.1007/978-3-642-39274-0_12
9. Donzé, A.: Breach, A toolbox for verification and parameter synthesis of hybrid systems. In: Touili, T., Cook, B., Jackson, P.B. (eds.) *Computer Aided Verification, 22nd International Conference, CAV 2010, Edinburgh, UK, July 15-19, 2010. Proceedings. Lecture Notes in Computer Science*, vol. 6174, pp. 167–170. Springer (2010). https://doi.org/10.1007/978-3-642-14295-6_17

10. Doveri, K., Ganty, P., Mazzocchi, N.: Forq-based language inclusion formal testing. In: Shoham, S., Vizel, Y. (eds.) *Computer Aided Verification - 34th International Conference, CAV 2022, Haifa, Israel, August 7-10, 2022, Proceedings, Part II*. Lecture Notes in Computer Science, vol. 13372, pp. 109–129. Springer (2022). https://doi.org/10.1007/978-3-031-13188-2_6
11. Hensel, C., Junges, S., Katoen, J., Quatmann, T., Volk, M.: The probabilistic model checker storm. *Int. J. Softw. Tools Technol. Transf.* **24**(4), 589–610 (2022). <https://doi.org/10.1007/S10009-021-00633-Z>, <https://doi.org/10.1007/s10009-021-00633-z>
12. Henzinger, T.A., Ho, P.: HYTECH: the cornell hybrid technology tool. In: Antsaklis, P.J., Kohn, W., Nerode, A., Sastry, S. (eds.) *Hybrid Systems II, Proceedings of the Third International Workshop on Hybrid Systems, Ithaca, NY, USA, October 1994*. Lecture Notes in Computer Science, vol. 999, pp. 265–293. Springer (1994). https://doi.org/10.1007/3-540-60472-3_14
13. Kwiatkowska, M.Z., Norman, G., Parker, D.: PRISM: probabilistic symbolic model checker. In: Field, T., Harrison, P.G., Bradley, J.T., Harder, U. (eds.) *Computer Performance Evaluation, Modelling Techniques and Tools 12th International Conference, TOOLS 2002, London, UK, April 14-17, 2002, Proceedings*. Lecture Notes in Computer Science, vol. 2324, pp. 200–204. Springer (2002). https://doi.org/10.1007/3-540-46029-2_13
14. Larsen, K.G., Pettersson, P., Yi, W.: UPPAAL in a nutshell. *Int. J. Softw. Tools Technol. Transf.* **1**(1-2), 134–152 (1997). <https://doi.org/10.1007/S100090050010>
15. Lombardy, S., Marsault, V., Sakarovitch, J.: Awali, a library for weighted automata and transducers (version 2.3) (2022), software available at <http://vaucanson-project.org/Awali/2.3/>
16. Lombardy, S., Poss, R., Régis-Gianas, Y., Sakarovitch, J.: Introducing VAUCANSON. In: Ibarra, O.H., Dang, Z. (eds.) *Implementation and Application of Automata, 8th International Conference, CIAA 2003, Santa Barbara, California, USA, July 16-18, 2003, Proceedings*. Lecture Notes in Computer Science, vol. 2759, pp. 96–107. Springer (2003). https://doi.org/10.1007/3-540-45089-0_10
17. Nickovic, D., Yamaguchi, T.: RTAMT: online robustness monitors from STL. In: Hung, D.V., Sokolsky, O. (eds.) *Automated Technology for Verification and Analysis - 18th International Symposium, ATVA 2020, Hanoi, Vietnam, October 19-23, 2020, Proceedings*. Lecture Notes in Computer Science, vol. 12302, pp. 564–571. Springer (2020). https://doi.org/10.1007/978-3-030-59152-6_34

A Antichain-Based Inclusion Checking for Quantitative Automata

We extended the inclusion algorithm of FORKLIFT [10] to support quantitative automata whose accepting states are drawn from nested automata. FORKLIFT's procedure for deciding $L(A) \subseteq L(B)$ between two Büchi automata A and B searches for counterexamples using membership queries. The approach prunes the search space by discarding candidates from $L(A)$ that are subsumed by other words of $L(A)$ with respect to a well-quasiorder. Termination follows from the mathematical properties of well-quasiorders, which guarantee that only finitely many candidates remain after pruning. Correctness is immediate: any kept candidate that violates $L(A) \subseteq L(B)$ witnesses the failure of inclusion. For completeness, the quasiorder must satisfy the following: for all candidates $w \in \Sigma^\omega$ subsumed by $w_0 \in \Sigma^\omega$, if $w_0 \in L(B)$ then $w \in L(B)$. Thus, if all kept candidates belong to $L(B)$, so do the discarded ones.

In [2], the algorithm was extended to LimSup automata by modifying the well-quasiorder used for pruning. We generalize it further to decide, for all $w \in \Sigma^\omega$, whether $w \in L(A) \Rightarrow (w \in L(B) \wedge A(w) \leq B(w))$. FORKLIFT relies on a *family of right quasiorders* (FORQ), defined using several well-quasiorders over finite words. This notion arises because inclusion of LimSup automata can be decided by reasoning solely on lasso words (i.e., words of the form uv^ω where $u \in \Sigma^*$ and $v \in \Sigma^+$). Our extension involves two generalizations: (1) the structural FORQ definition from [10], and (2) the membership query.

Definition 1. Let $\mathcal{B} = (\Sigma, Q, F, \iota, \delta)$ be a LimSup automaton with accepting states F and weights $W = \{\gamma(t) \mid t \text{ is a transition of } \mathcal{B}\}$. We generalize the structural FORQ of $\mathcal{B}$ from [10] to the pair $(\preceq^\mathcal{B}, \{\preceq_u^\mathcal{B}\}_{u \in \Sigma^*})$ where:

$$\begin{aligned} u_1 \preceq^\mathcal{B} u_2 &\iff \text{Tgt}_\mathcal{B}(u_1) \subseteq \text{Tgt}_\mathcal{B}(u_2) \\ v_1 \preceq_u^\mathcal{B} v_2 &\iff \text{Cxt}_\mathcal{B}(\text{Tgt}_\mathcal{B}(u), v_1) \subseteq \text{Cxt}_\mathcal{B}(\text{Tgt}_\mathcal{B}(u), v_2) \end{aligned}$$

with $\text{Tgt}_\mathcal{B}: \Sigma^* \rightarrow 2^Q$ and $\text{Cxt}_\mathcal{B}: 2^Q \times \Sigma^+ \rightarrow 2^{Q \times Q \times W \times \{\top, \perp\}}$ defined by:

$$\begin{aligned} \text{Tgt}_\mathcal{B}(u) &= \{q' \in Q \mid \iota \xrightarrow{u}_\mathcal{B} q'\} \\ \text{Cxt}_\mathcal{B}(S, v) &= \left\{ (q, q', x, b) \mid \begin{array}{l} q \in S, \rho = q \xrightarrow{v}_\mathcal{B} q', \\ x \text{ is the maximal weight visited by } \rho, \\ b = \top \text{ iff } \rho \text{ visits an accepting state} \end{array} \right\} \end{aligned}$$

Extending the properties of structural FORQ established by [10] to this definition is straightforward. However, our inclusion algorithm requires different membership queries than both [10] and [2]. Given a lasso word uv^ω accepted by A , the algorithm must determine whether B admits an accepting run ρ on uv^ω with value at least $A(uv^\omega)$. We handle this via an on-the-fly membership query for generalized Büchi automata.

B Examples from the Benchmarks

We demonstrate our benchmark families with concrete examples. Figure 3 shows $\mathbb{A}_{2,2}$, which tolerates at most two pending requests that must be granted within two events. Figure 4 shows $\mathbb{B}_{1,2}$ with one process and two resources; the child automaton uses four states to track the powerset $2^{\{a_1, a_2\}}$ of accessed resources.

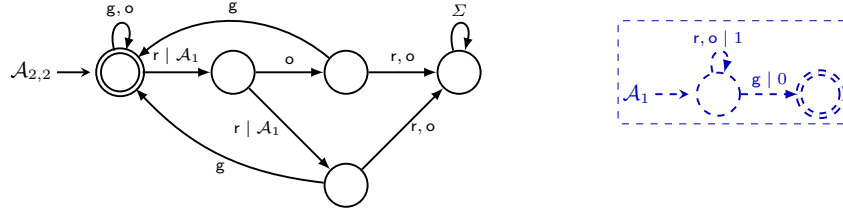


Fig. 3: The response-time automaton $\mathbb{A}_{2,2} = (\mathcal{A}_{2,2}, \mathcal{A}_1)$. Every parent transition without a child automaton annotation is silent.

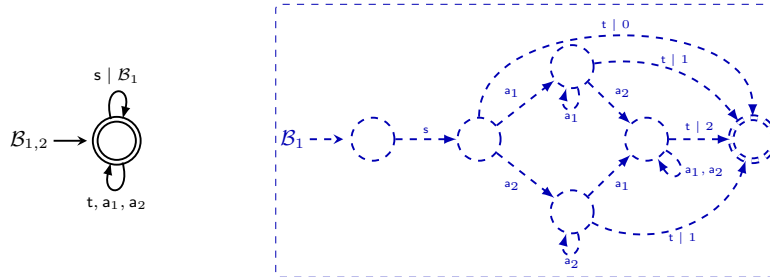


Fig. 4: The resource-consumption automaton $\mathbb{B}_{1,2} = (\mathcal{B}_{1,2}, \mathcal{B}_1)$. Every parent transition without a child automaton annotation is silent. The rejecting sink state of the child automaton is not shown. Every child transition without a weight annotation has weight 0.

C Additional Data from the Experiments

Table 4 reports the sizes of the flattened automata constructed during our experiments. These statistics correspond to the largest parameter values for which each benchmark completed within the resource limits.

Benchmark	n	k	States	Edges	SCCs
<i>Response Time</i>					
(Sup; Sum ⁺) emptiness	256	256	5,691,274	17,204,904	5,625,478
(LimSupAvg; Sum ⁺) emptiness	7	10	14,078,497	14,441,408	14,076,517
(Sup; Sum ^B) universality	8	9	3,664,092	4,132,360	3,663,070
<i>Resource Consumption</i>					
(Sup; Max) emptiness	4	3	558,024	11,218,554	36,904
(LimSupAvg; Max) emptiness	2	4	5,829	4,601,278	2

Table 4: Statistics of flattened automata at maximum solved parameters.

Table 4 suggests that the hard instances differ mainly in graph shape, not just in state count. The response-time flattenings are sparse, due to the benchmark’s small alphabet, and have SCC counts close to the state counts, so most SCCs are singletons and only a fraction of states sit in nontrivial cycles. The resource-consumption flattenings are far denser as expected: (Sup, Max) already has about 20 edges per state, and the (LimSupAvg, Max) maximum solved instance has 4.6M edges for only 5,829 states and just 2 SCCs, so it is essentially one large component. This explains why the resource consumption benchmarks become intractable quickly; density and cyclic structure dominate even before the state space grows large.